

Siguiendo estrictamente las directrices del proyecto y manteniendo la arquitectura base propuesta en la etapa anterior, he procedido a implementar el sistema completo de escenas utilizando la Máquina de Estados Finitos (FSM).

Al no requerir lógica de enemigos ni torres por el momento, la simulación de transición a la pantalla de "Game Over" se ha mapeado a una tecla de depuración temporal.

A continuación, presento la entrega del sistema de escenas.

1. Estructura de Carpetas Actualizada

No se ha creado ningún archivo nuevo para evitar romper la arquitectura establecida. Todo el diseño se ha integrado y escalado sobre los módulos principales del núcleo (Core).

Archivos Modificados:

- `src/core/state_manager.py` (Se añadieron los nuevos estados y la lógica de navegación).
- `src/core/game_manager.py` (Se registraron los nuevos estados en el motor principal).

```
el_ultimo_bastion/
├── config/
│   └── settings.json
├── src/
│   ├── core/
│   │   ├── __init__.py
│   │   ├── game_manager.py [MODIFICADO]
│   │   └── state_manager.py [MODIFICADO]
│   └── __init__.py
├── main.py
└── requirements.txt
```

2. Código Completo

`src/core/state_manager.py`

```

import pygame

class State:
    """Clase abstracta pura para definir el comportamiento de los estados."""
    def __init__
        self.manager = state_manager

    def on_enter
        """Se ejecuta al entrar en este estado. Útil para reiniciar variables."""
        pass

    def on_exit
        """Se ejecuta al salir de este estado."""
        pass

    def handle_events
        pass

    def update
        pass

    def draw
        pass

    def _draw_menu_options
        """Método auxiliar protegido para renderizar menús interactivos."""
        font = pygame.font.SysFont("Arial", 40, bold=True)
        for i, option in enumerate(options):
            if i == selected_index:
                # Opción seleccionada: Color Dorado
                text_surface = font.render(f"> {option} <", True, (212, 175, 55))
            else:
                # Opción inactiva: Color Gris
                text_surface = font.render(option, True, (150, 150, 150))

        # Centrar el texto en la pantalla
        rect = text_surface.get_rect(center=(screen.get_width() // 2, start_y + (i * 60)))
        screen.blit(text_surface, rect)

class MenuState(State):
    """Estado: Menú Principal"""
    def __init__

```



```

def draw
    screen.fill((34, 47, 34)) # Fondo terreno verde militar

    font = pygame.font.SysFont("Arial", 32)
    text1 = font.render("ESTADO: JUGANDO", True, (240, 240, 240))
    text2 = font.render("[ESC] Pausar juego", True, (150, 250, 150))
    text3 = font.render("[G] Simular Game Over (Depuración)", True, (250, 150, 150))

    screen.blit(text1, (50, 50))
    screen.blit(text2, (50, 100))
    screen.blit(text3, (50, 150))

```

```

class PauseState(State):
    """Estado: Juego en Pausa"""
    def __init__(
        self, state_manager
    ):
        self.options = ["Continuar", "Volver al Menú"]
        self.selected_index = 0

    def on_enter
        self.selected_index = 0

    def handle_events
        for event in events:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_UP:
                    self.selected_index = max(0, self.selected_index - 1)
                elif event.key == pygame.K_DOWN:
                    self.selected_index = min(len(self.options) - 1, self.selected_index + 1)
                elif event.key == pygame.K_RETURN:
                    if self.selected_index == 0:
                        self.manager.change_state("GAME")
                    elif self.selected_index == 1:
                        self.manager.change_state("MENU")
                elif event.key == pygame.K_ESCAPE:
                    # Permite salir de la pausa también con ESC
                    self.manager.change_state("GAME")

```

```

def draw
    # No limpiamos la pantalla (screen.fill) para dejar el juego de fondo
    # Dibujamos un overlay translúcido (requiere Surface con canal Alpha)

```

```

overlay = pygame.Surface((screen.get_width(), screen.get_height()),
pygame.SRCALPHA)
overlay.fill((0, 0, 0, 180)) # Negro al 70% de opacidad
screen.blit(overlay, (0, 0))

title_font = pygame.font.SysFont("Arial", 54, bold=True)
title_text = title_font.render("PAUSA", True, (255, 255, 255))
title_rect = title_text.get_rect(center=(screen.get_width() // 2, 200))
screen.blit(title_text, title_rect)

self._draw_menu_options(screen, self.options, self.selected_index, 350)

```

```

class GameOverState(State):
    """Estado: Derrota del jugador"""
    def __init__(
        self,
        state_manager
    ):
        self.options = ["Reiniciar", "Salir"]
        self.selected_index = 0

    def on_enter(
        self
    ):
        self.selected_index = 0

    def handle_events(
        self
    ):
        for event in events:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_UP:
                    self.selected_index = max(0, self.selected_index - 1)
                elif event.key == pygame.K_DOWN:
                    self.selected_index = min(len(self.options) - 1, self.selected_index + 1)
                elif event.key == pygame.K_RETURN:
                    if self.selected_index == 0:
                        # Al reiniciar, volvemos a GAME. El método on_enter de GAME
                        # en el futuro se encargará de resetear las oleadas y vida.
                        self.manager.change_state("GAME")
                    elif self.selected_index == 1:
                        self.manager.quit_game()

    def draw(
        self
    ):
        screen.fill((50, 10, 10)) # Fondo rojo oscuro (sangre/fuego)

        title_font = pygame.font.SysFont("Arial", 64, bold=True)
        title_text = title_font.render("EL BASTIÓN HA CAÍDO", True, (255, 50, 50))

```

```

title_rect = title_text.get_rect(center=(screen.get_width() // 2, 200))
screen.blit(title_text, title_rect)

self._draw_menu_options(screen, self.options, self.selected_index, 350)

```

```

class StateManager:
    """Máquina de Estados Finitos (FSM)"""
    def __init__
        self.states = {}
        self.current_state = None
        self.quit_requested = False # Bandera de control global

    def add_state
        self.states[name] = state

    def change_state
        if self.current_state:
            self.current_state.on_exit() # Dispara limpieza del estado anterior

        if name in self.states:
            self.current_state = self.states[name]
            self.current_state.on_enter() # Inicializa el nuevo estado

    def quit_game
        """Solicita el cierre seguro al Game Loop principal."""
        self.quit_requested = True

```

src/core/game_manager.py

```

import pygame
import json
import os
import sys
from src.core.state_manager import StateManager, MenuState, GameState, PauseState,
GameOverState

```

```

class GameManager:
    def __init__
        pygame.init()
        self.load_configuration()

        self.width = self.config["screen"]["width"]
        self.height = self.config["screen"]["height"]
        self.screen = pygame.display.set_mode((self.width, self.height))
        pygame.display.set_caption(self.config["screen"]["title"])

        self.clock = pygame.time.Clock()
        self.target_fps = self.config["screen"]["fps"]
        self.is_running = True

        # Instanciación y registro de TODAS las escenas
        self.state_manager = StateManager()
        self.state_manager.add_state("MENU", MenuState(self.state_manager))
        self.state_manager.add_state("GAME", GameState(self.state_manager))
        self.state_manager.add_state("PAUSE", PauseState(self.state_manager))
        self.state_manager.add_state("GAME_OVER", GameState(self.state_manager))

        # Estado inicial
        self.state_manager.change_state("MENU")

    def load_configuration
        config_path = os.path.join("config", "settings.json")
        try:
            with open(config_path, "r", encoding="utf-8") as file:
                self.config = json.load(file)
        except (FileNotFoundError, json.JSONDecodeError):
            self.config = {
                "screen": {"width": 1280, "height": 720, "title": "El Ultimo Bastion", "fps": 60}
            }

    def run
        # El bucle ahora respeta la bandera de salida de la FSM
        while self.is_running and not self.state_manager.quit_requested:
            dt = self.clock.tick(self.target_fps) / 1000.0

            self.handle_events()
            self.update(dt)
            self.draw()

```

```

def handle_events
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            self.is_running = False

    if self.state_manager.current_state:
        self.state_manager.current_state.handle_events(events)

def update
    if self.state_manager.current_state:
        self.state_manager.current_state.update(dt)

def draw
    # NOTA: No hacemos fill() a nivel general aquí para permitir
    # que estados como "PAUSA" preserven el renderizado anterior.
    # Es responsabilidad de cada estado limpiar la pantalla si lo requiere.
    if getattr(self.state_manager.current_state, "__class__", None).__name__ != "PauseState":
        self.screen.fill((0, 0, 0))

    if self.state_manager.current_state:
        self.state_manager.current_state.draw(self.screen)

    pygame.display.flip()

def terminate
    pygame.quit()
    sys.exit()

```

3. Explicación Técnica de la Implementación

1. **Ciclo de Vida de los Estados (on_enter / on_exit):** Se agregaron métodos base a la clase State. Esto es fundamental en videojuegos de este calibre. Por ejemplo, cuando se llama a `change_state("MENU")`, el método `on_enter` reinicia el cursor `self.selected_index = 0` para asegurar que el jugador siempre inicie en la primera opción, sin importar dónde dejó el cursor la vez anterior.
2. **Sistema de Menús e Índices:** La navegación no usa el mouse para evitar colisiones lógicas complejas en esta etapa. Utiliza un índice matemático (`self.selected_index`). Al presionar las flechas ARRIBA/ABAJO, el valor suma o resta limitándose entre 0 y la longitud máxima del arreglo de opciones (`max()` y `min()`), garantizando que el cursor no provoque un error `IndexError`.
3. **Transparencia (Alpha Blending) en la Pausa:**

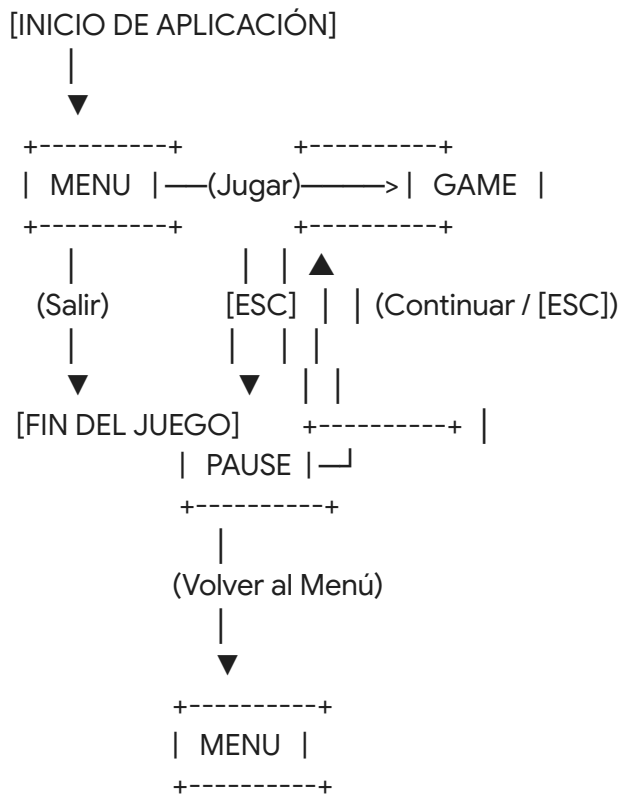
El PauseState no limpia la pantalla completa. En su lugar, crea una `pygame.Surface` con soporte de canal Alfa (`SRCALPHA`), la llena de negro con un nivel de transparencia de 180 (donde 255 es sólido) y la dibuja sobre el juego. En el `GameManager`, se omitió la limpieza global `screen.fill(0,0,0)` exclusivamente cuando el estado activo es la pausa, logrando el efecto visual de un "Overlay".

4. Desacoplamiento de Salida (`quit_requested`):

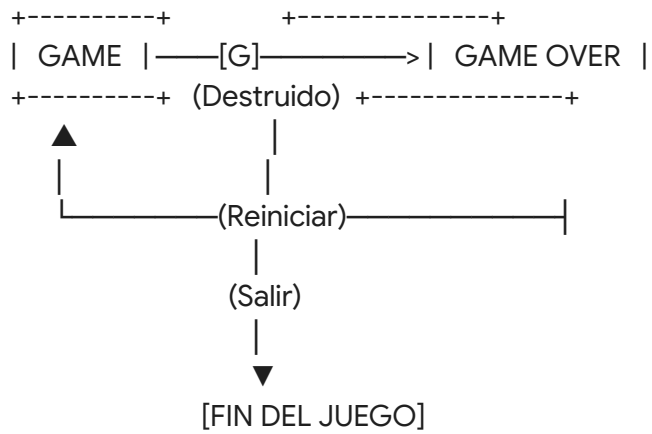
Los estados no fuerzan el cierre abrupto usando `sys.exit()` porque eso corrompería la memoria y rompería el entorno de pruebas. En su lugar, usan un *Event Flag* (`self.manager.quit_game()`). El `GameManager` monitorea esta bandera en la condición del ciclo `while` principal para realizar un cierre limpio (Graceful Exit).

4. Flujo Textual de Navegación (Máquina de Estados)

A continuación se esquematiza el grafo dirigido que describe el comportamiento lógico entre las escenas que puedes probar en el código:



(Línea alternativa por evento interno del Gameplay)



Fuentes

1. <https://github.com/Hardcoded-inc/genetic-racer>
2. <https://github.com/Hardcoded-inc/genetic-racer>
3. <https://github.com/Hardcoded-inc/genetic-racer>
4. <https://github.com/Hardcoded-inc/genetic-racer>